

PASSWORD CAPTURING USING WIRESHARK

Objective: Demonstrate how packet captures can expose information transmitted through an unencrypted HTTP connection, using Wireshark in an authorized laboratory environment.

Important: Perform packet capture only on networks, devices, and applications for which you have permission. HTTPS/TLS is designed to prevent credentials from appearing as readable HTTP form data.

1. Introduction

Wireshark is a network protocol analyzer capable of displaying packets captured from a network interface. For unencrypted protocols such as HTTP, captured application data may be visible in packet details. This report demonstrates the analysis workflow shown in the supplied screenshots: start a capture, generate web traffic, filter HTTP packets, distinguish GET and POST requests, and inspect the submitted form data.

Workflow demonstrated

1. Start Wireshark and capture traffic 2. Open the demonstration login page 3. Filter HTTP traffic 4. Check GET requests 5. Check POST requests 6. Expand the HTML form URL encoded section to inspect submitted fields.

Useful display filters shown in the screenshots

```
http
http.request.method == "GET"
http.request.method == "POST"
```

The screenshots below are the primary evidence for each stage; they are intentionally retained as screenshots rather than replaced with generic diagrams.

Step 1 — Start Wireshark packet capture

The image shows the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. Below the menu is a toolbar with various icons for file operations, capture control, and analysis. The main window is divided into three panes. The top pane shows a list of captured packets with columns for No., Time, Source, Destination, Protocol, Length, and Info. The bottom pane shows the detailed view of the selected packet (No. 377), including Ethernet II, Internet Protocol Version 6, User Datagram Protocol, and Data (78 bytes). The Data field is expanded, showing a hexadecimal dump and its corresponding ASCII representation.

No.	Time	Source	Destination	Protocol	Length	Info
377	15.530221	2409:40f4:10fd:fa82...	64:ff9b::14b8:af0b	TCP	86	[TCP Dup ACK 373#1] 60070 → 443 [ACK] Seq=11427 Ack=10077 Win=260864 Len=0 SLE=9586 SRE=10077
378	15.640045	64:ff9b::14b8:af0b	2409:40f4:10fd:fa82...	TCP	76	443 → 60070 [ACK] Seq=10077 Ack=9027 Win=4194560 Len=0
379	15.652174	64:ff9b::14b8:af0b	2409:40f4:10fd:fa82...	TCP	76	443 → 60070 [ACK] Seq=10077 Ack=11427 Win=4194560 Len=0
380	15.683243	64:ff9b::14b8:af0b	2409:40f4:10fd:fa82...	TLSv1.3	519	Application Data
381	15.683347	2409:40f4:10fd:fa82...	64:ff9b::14b8:af0b	TCP	74	60070 → 443 [ACK] Seq=11427 Ack=10522 Win=262144 Len=0

> Frame 1: 140 bytes on wire (1120 bits), 140 bytes captured (1120 bits) on interface \Device\NPF_{B049A020-DB5D-40FE-B845-853876C5B588}, id 0
> Ethernet II, Src: 56:b3:f6:a1:06:ed (56:b3:f6:a1:06:ed), Dst: e0:8f:4c:55:4e:66 (e0:8f:4c:55:4e:66)
> Internet Protocol Version 6, Src: 2404:6800:4007:83c::200a, Dst: 2409:40f4:10fd:fa82:11df:efa3:d588:8e55
> User Datagram Protocol, Src Port: 443, Dst Port: 55012
> Data (78 bytes)

```
0000 e0 8f 4c 55 4e 66 5b f6 a1 06 ed 86 dd 68 00  --LUNFV- - - - -h-
0010 00 00 00 56 11 39 24 04 68 00 40 07 08 3c 00 00  --V-9$- h-@- - - -
0020 00 00 00 00 20 0a 24 09 40 f4 10 fd fa 82 11 df  -- - $- @- - - -
0030 ef a3 d5 88 8e 55 01 bb d6 e4 00 56 5d 5c 5b 6a  -- -U- - - -V]\[j
0040 82 c8 18 1e d6 0b bb ae 3d ea cc 64 5e b1 64 92  -- - - - - = - d ^ - d -
0050 83 90 55 65 6e a6 b4 70 2a 41 74 05 65 c0 d9 cf  --Uen- -p *At-e- - -
0060 8f 61 8e 11 5a db a2 4b e4 ca 62 52 a8 eb 9e 33  --a- -Z- -K- -bR- - -3
0070 8e a9 bb c7 06 11 9b f8 b2 57 e5 28 bc 19 df b6  -- - - - - -W- ( - - -
0080 62 37 1f da 97 b0 f5 f7 b0 0d 47 29 b7 - - - - -G)
```

Wi-Fi: <live capture in progress> | Packets: 381 · Displayed: 381 (100.0%) | Profile: Default

Figure 1. Step 1 — Start Wireshark packet capture.

The capture is running on the Wi-Fi interface. Packets are being collected before and during the web activity.

Step 2 — Login page and submitted credentials

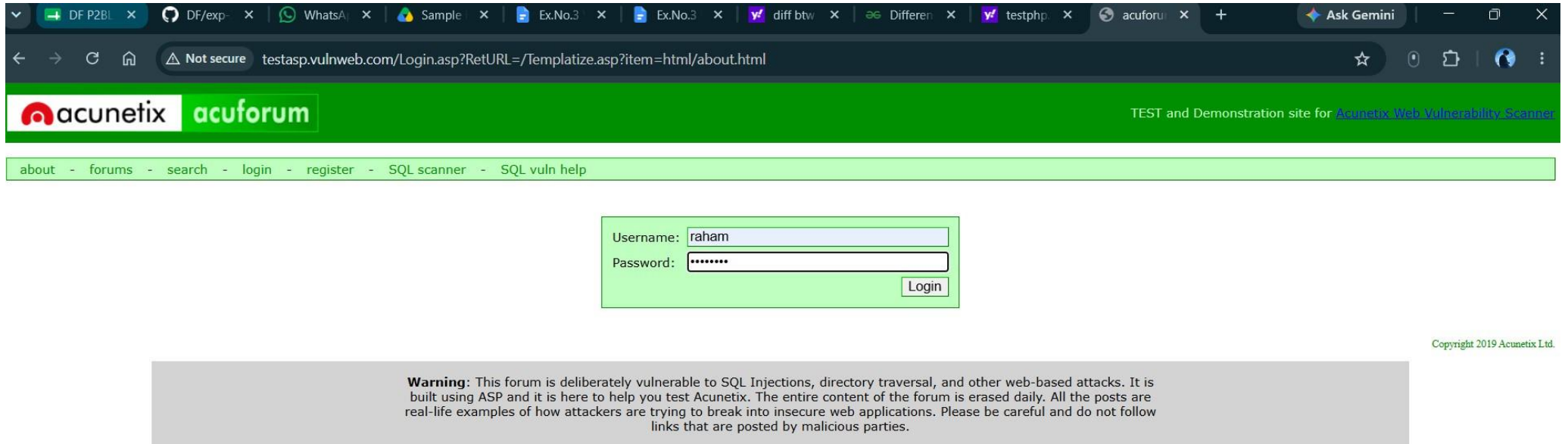


Figure 2. Step 2 — Login page and submitted credentials.

The demonstration website displays a login form.

Step 3/4 — Filtering captured traffic for HTTP

The image shows the Wireshark network protocol analyzer interface. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. Below the menu is a toolbar with various icons. The main display area is divided into three panes. The top pane shows a list of captured packets, with a filter 'http' applied in the top-left corner. The list shows several HTTP GET requests to a filestreamingservice. The middle pane shows the details of the selected packet (Frame 983), including Ethernet II, Internet Protocol Version 6, and Hypertext Transfer Protocol. The bottom pane shows the raw packet data in hexadecimal and ASCII.

No.	Filter	Destination	Protocol	Length	Info
14	http.request.method == "POST"	98.70.194.73	HTTP	535	GET /filestreamingservice/files/bcf25e2c-3271-4385-9051-15d05da4b470?P1=1786212206&P2=404&P3=2&P4=KCz5vxkyAuy2wjQc%2b4QQdwReDcqKO...
15	http.request.method == "GET"	10.93.214.196	HTTP	1024	HTTP/1.1 206 Partial Content
16	http	98.70.194.73	HTTP	536	GET /filestreamingservice/files/bcf25e2c-3271-4385-9051-15d05da4b470?P1=1786212206&P2=404&P3=2&P4=KCz5vxkyAuy2wjQc%2b4QQdwReDcqKO...
17	http2	98.70.194.73	HTTP	533	GET /filestreamingservice/files/75bd5c2b-4555-4259-8953-51eb8edfe2a6?P1=1786207239&P2=404&P3=2&P4=gP8IGDZM3JvwN39w9fBW2KkEW4emOy1...
18		98.70.194.73	HTTP	533	GET /filestreamingservice/files/75bd5c2b-4555-4259-8953-51eb8edfe2a6?P1=1786207239&P2=404&P3=2&P4=gP8IGDZM3JvwN39w9fBW2KkEW4emOy1...

Frame 983: 186 bytes on wire (1488 bits), 186 bytes captured (1488 bits) on interface \Device\NPF_{B049A020-DB5D-40FE-B845-853876C5B588}, id 0

Ethernet II, Src: e0:8f:4c:55:4e:66 (e0:8f:4c:55:4e:66), Dst: 56:b3:f6:a1:06:ed (56:b3:f6:a1:06:ed)

Internet Protocol Version 6, Src: 2409:40f4:10fd:fa82:11df:efa3:d588:8e55, Dst: 2405:200:1630:a03::312c:c5b9

Transmission Control Protocol, Src Port: 60072, Dst Port: 80, Seq: 1, Ack: 1, Len: 112

Hypertext Transfer Protocol

```
0000 56 b3 f6 a1 06 ed e0 8f 4c 55 4e 66 86 dd 60 06 V..... LUNf..
0010 1a 5f 00 84 06 fe 24 09 40 f4 10 fd fa 82 11 df ._.$. @.....
0020 ef a3 d5 88 8e 55 24 05 02 00 16 30 0a 03 00 00 ....U$. ...0...
0030 00 00 31 2c c5 b9 ea a8 00 50 4e 3e ce 8a 78 da --1,.... -PN>..x.
0040 bd 0f 50 18 00 ff 13 87 00 00 47 45 54 20 2f 63 --P..... -GET /c
0050 6f 6e 6e 65 63 74 74 65 73 74 2e 74 78 74 20 48 onnectte st.txt H
0060 54 54 50 2f 31 2e 31 0d 0a 43 6f 6e 6e 65 63 74 TTP/1.1 -Connect
0070 69 6f 6e 3a 20 43 6c 6f 73 65 0d 0a 55 73 65 72 ion: Clo se..User
0080 2d 41 67 65 6e 74 3a 20 4d 69 63 72 6f 73 6f 66 -Agent: Microsof
0090 74 20 4e 43 53 49 0d 0a 48 6f 73 74 3a 20 69 70 t NCSI.. Host: ip
00a0 76 36 2e 6d 73 66 74 63 6f 6e 6e 65 63 74 74 65 v6.msftc onnectte
00b0 73 74 2e 63 6f 6d 0d 0a 0d 0a st.com.. ..
```

Figure 3. Step 3/4 — Filtering captured traffic for HTTP.

The display filter `http` is used to narrow the packet list to HTTP traffic.

Step 5/6 — Inspecting HTTP GET requests

The image shows a Wireshark packet capture window with the filter `http.request.method == GET` applied. The packet list displays several HTTP GET requests. The selected packet (No. 983) is expanded, showing the Ethernet II, Internet Protocol Version 6, and Hypertext Transfer Protocol layers. The packet details pane shows the raw packet data in hexadecimal and ASCII.

No.	Time	Source	Destination	Protocol	Length	Info
983	68.959685	2409:40f4:10fd:fa82...	2405:200:1630:a03::...	HTTP	186	GET /connecttest.txt HTTP/1.1
984	68.959826	2409:40f4:10fd:fa82...	64:ff9b::312c:c5e2	HTTP	185	GET /connecttest.txt HTTP/1.1
2529	268.206868	10.93.214.196	98.70.194.73	HTTP	350	/filestreamingservice//files/37d5e003-75e0-4cd0-a214-6d38db092247/pieceshash?cacheHostOrigin=dl.delivery.mp.microsoft.com HTTP...
2574	268.543306	10.93.214.196	98.70.194.73	HTTP	534	/filestreamingservice/files/37d5e003-75e0-4cd0-a214-6d38db092247?P1=1786206900&P2=404&P3=2&P4=BhBYc8PU%2f%2bUanfHfEySHAD7fE0gR...
2971	270.521588	10.93.214.196	98.70.194.73	HTTP	350	/filestreamingservice//files/bcf25e2c-3271-4385-9051-15d05da4b470/pieceshash?cacheHostOrigin=dl.delivery.mp.microsoft.com HTTP...

Frame 983: 186 bytes on wire (1488 bits), 186 bytes captured (1488 bits) on interface \Device\NPF_{B049A020-DB5D-40FE-B845-853876C5B588}, id 0

Ethernet II, Src: e0:8f:4c:55:4e:66 (e0:8f:4c:55:4e:66), Dst: 56:b3:f6:a1:06:ed (56:b3:f6:a1:06:ed)

Internet Protocol Version 6, Src: 2409:40f4:10fd:fa82:11df:efa3:d588:8e55, Dst: 2405:200:1630:a03::312c:c5b9

Transmission Control Protocol, Src Port: 60072, Dst Port: 80, Seq: 1, Ack: 1, Len: 112

Hypertext Transfer Protocol

```
0000 56 b3 f6 a1 06 ed e0 8f 4c 55 4e 66 86 dd 60 06 V..... LUNf...
0010 1a 5f 00 84 06 fe 24 09 40 f4 10 fd fa 82 11 df _....$. @.....
0020 ef a3 d5 88 8e 55 24 05 02 00 16 30 0a 03 00 00 ....U$. ...0....
0030 00 00 31 2c c5 b9 ea a8 00 50 4e 3e ce 8a 78 da --1,....-PN>...x
0040 bd 0f 50 18 00 ff 13 87 00 00 47 45 54 20 2f 63 --P.....-GET /c
0050 6f 6e 6e 65 63 74 74 65 73 74 2e 74 78 74 20 48 onnectte st.txt H
0060 54 54 50 2f 31 2e 31 0d 0a 43 6f 6e 6e 65 63 74 TTP/1.1 -Connect
0070 69 6f 6e 3a 20 43 6c 6f 73 65 0d 0a 55 73 65 72 ion: Clo se..User
0080 2d 41 67 65 6e 74 3a 20 4d 69 63 72 6f 73 6f 66 -Agent: Microsof
0090 74 20 4e 43 53 49 0d 0a 48 6f 73 74 3a 20 69 70 t NCSI.. Host: ip
00a0 76 36 2e 6d 73 66 74 63 6f 6e 6e 65 63 74 74 65 v6.msftc onnectte
00b0 73 74 2e 63 6f 6d 0d 0a 0d 0a st.com..
```

Figure 4. Step 5/6 — Inspecting HTTP GET requests.

The GET filter shows HTTP GET requests. A GET request does not necessarily contain the login form submission; the POST request is checked next.

Step 7 — Inspecting the HTTP POST form data

The image shows a Wireshark packet capture window. The top menu bar includes File, Edit, View, Go, Capture, Analyze, Statistics, Telephony, Wireless, Tools, and Help. The toolbar contains various icons for packet capture and analysis. The packet list pane shows a single packet selected: No. 34811, Time 462.701715, Source 2409:40f4:10fd:fa82..., Destination 64:ff9b::2cee:1df4, Protocol HTTP, Length 869, Info POST /Login.asp?RetURL=/Templatize.asp?item=html/about.html HTTP/1.1 (application/x-www-form-urlencoded). The packet details pane shows the following structure:

- [Response in frame: 34931]
- [Next request in frame: 34937]
- File Data: 30 bytes
- HTML Form URL Encoded: application/x-www-form-urlencoded
 - Form item: "tfUName" = "raham"
 - Form item: "tfUPass" = "12345678"

The packet bytes pane displays the raw data in hexadecimal and ASCII. The ASCII column shows the following text:

```
V.....LUNf...  
.../..$.@.....  
...U..d.....  
...P=.../  
;..P...#...POST /  
Login.asp?RetURL  
=/Templa tize.asp  
?item=ht ml/about  
.html HT TP/1.1..  
Host: te stasp.vu  
Inweb.co m...Conne  
ction: k eep-aliv  
e...Conte nt-Lengt  
h: 30...C ache-Con  
trol: ma x-age=0..  
..Upgrade -Insecur  
e-Reques ts: 1...C  
ontent-T ype: app  
lication /x-www-f  
orm-urle ncoded..  
User-Age nt: Mozi  
lla/5.0 (Windows  
NT 10.0 ; Win64;  
x64) Ap pleWebKi  
t/537.36 (KHTML,  
like Ge cko) Chr  
ome/151.0.0.0 Sa  
fari/537 .36...Ori  
gin: htt p://test
```

The status bar at the bottom shows: wireshark Wi-Fi 20260808214432 a20824.pcapng | Packets: 53117 · Disposed: 1 (0.0%) | Profile: Default

Figure 5. Step 7 — Inspecting the HTTP POST form data.

The POST packet is selected and the HTML Form URL Encoded section exposes the submitted form fields. This is possible because the demonstration traffic is plain HTTP rather than encrypted HTTPS.

2. Result and Conclusion

The supplied capture demonstrates that Wireshark can reveal application-layer information when that information is transmitted over an unencrypted protocol such as HTTP. By filtering the capture with HTTP, GET, and POST display filters, the relevant login transaction can be located and its form-encoded fields inspected.

Security takeaway: Credentials should never be transmitted over plain HTTP. Web applications should enforce HTTPS/TLS, use secure session management, and protect sensitive information in transit.

Evidence included

All five supplied screenshots are included in their original visual form ...